

改进后的完整软件过程定义文档 —— HarmonyWardrobe

版本历史

版本	日期	修改内容	作者
V1.0	2026年春	初始过程定义（基于ISO/IEC 12207/15504/33001剪裁）	项目组
V2.0	2026年6月	融入AI技术改进——新增AI角色、AI工具节点、AI度量指标	项目组

第1章 过程概述与目标

1.1 项目背景

HarmonyWardrobe是基于HarmonyOS开发的AI智能衣柜系统，核心功能包括衣物数字化管理、AI穿搭推荐、穿搭效果图合成与3D穿搭展示。需求划分为T0核心功能与T1迭代功能。本过程文档定义了涵盖AI技术融入后的全生命周期软件过程体系。

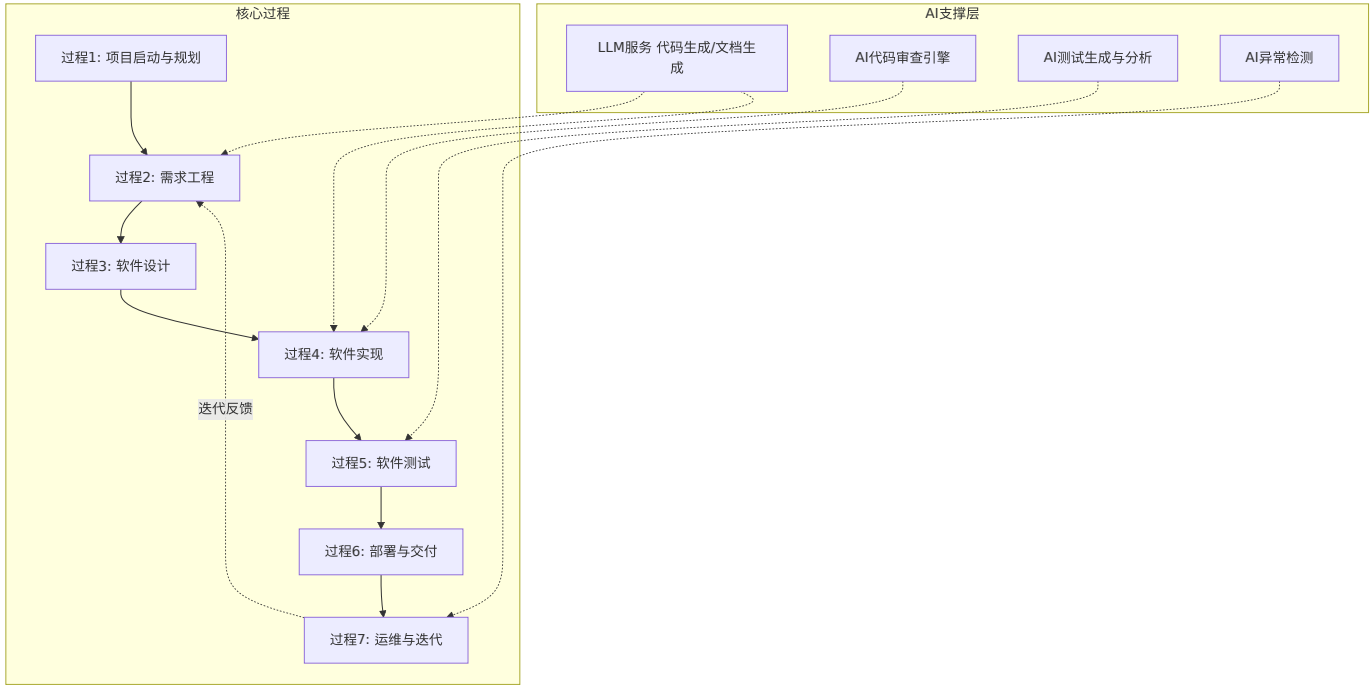
1.2 过程设计依据

本过程体系的设计依据包括五项标准与报告。ISO/IEC 12207提供了软件生命周期过程的主体框架。ISO/IEC 15504（SPICE）确立了过程质量与评估的标准。ISO/IEC 33001规范了过程治理与持续改进的要求。此外，AI4SE行业现状调查报告（2024）和中国信通院发布的《智能化软件工程技术和应用要求第3部分：智能测试能力》（2024）为AI技术的融入提供了行业参考。

1.3 过程改进目标

本次过程改进设定了四项核心目标。第一，将AI技术系统化融入软件全生命周期，实现人机协同的智能开发范式。第二，在需求工程、软件实现、软件测试等环节借助AI工具实现效率倍增。第三，建立AI输出物的质量标准与验证机制，确保AI引入后的交付质量。第四，构建可度量的AI贡献度指标体系，量化评估AI技术的过程改进效果。

1.4 过程体系架构



第2章 角色职责定义

2.1 角色总览

角色名称	来源	参与过程	职责概述
项目经理	原有	全部	项目整体管理与协调
需求分析师	原有	P1, P2	需求获取、分析与建模
架构师	原有	P3	系统架构设计与技术决策
开发工程师	原有	P4	编码实现、单元测试
测试工程师	原有	P5	测试设计与执行
运维工程师	原有	P6, P7	部署、运维与监控
AI训练师	新增	P2, P4, P5	Prompt工程、AI知识库维护
AI测试分析师	新增	P5	AI测试工具配置与输出审核
AI输出审核员	新增	P2, P4, P5	AI生成物质量验证

2.2 新增角色详细定义

AI训练师由一名开发工程师兼任，其核心职责包括设计并优化各环节AI工具的Prompt模板、维护项目级上下文知识库（涵盖代码库、设计文档和历史用例），以及持续跟踪AI输出质量并迭代Prompt策略。该角色的能力要求包括熟悉大语言模型的使用技巧、具备基本的Prompt工程能力并理解软件工程各阶段的活动。其交付物为《AI Prompt模板库》和《AI工具使用规范》。

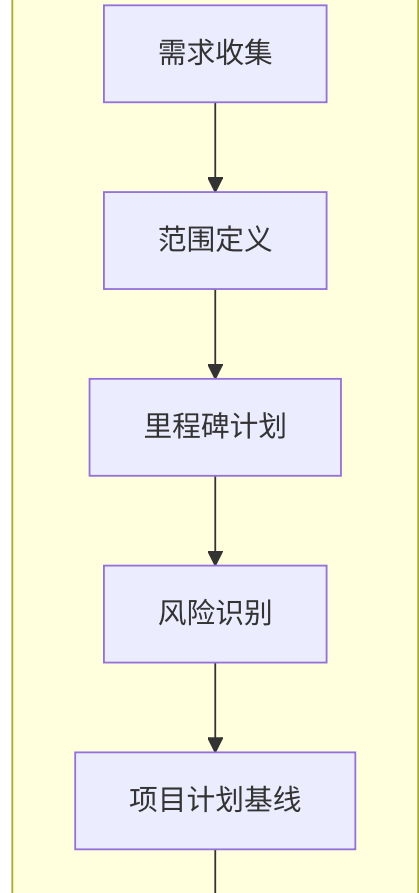
AI测试分析师由测试工程师兼任，其核心职责包括配置AI测试生成工具参数、审核AI生成的测试用例质量、管理缺陷预测模型以及跟踪AI测试覆盖率指标。该角色的能力要求包括熟悉软件测试方法论、了解AI测试工具原理并具备测试数据管理能力。其交付物为《AI测试用例审核报告》和《AI测试覆盖率分析》。

AI输出审核员由各阶段负责人兼任，需求阶段由需求分析师承担，编码阶段由开发工程师承担，测试阶段由测试工程师承担。其核心职责是对AI生成的所有输出物进行质量审核，确保AI输出符合项目质量标准，并对AI输出中的幻觉或错误进行标记和修正。该角色的能力要求为具备对应阶段的专业评审能力，其交付物为《AI输出审核记录》。

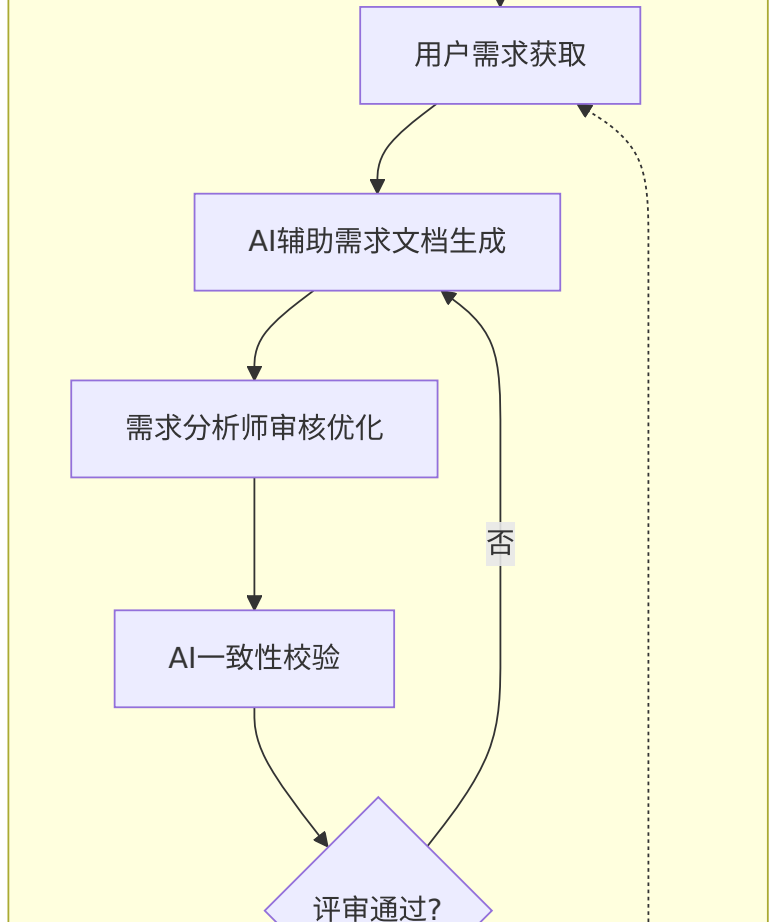
第3章 各阶段活动流程图（含AI参与节点）

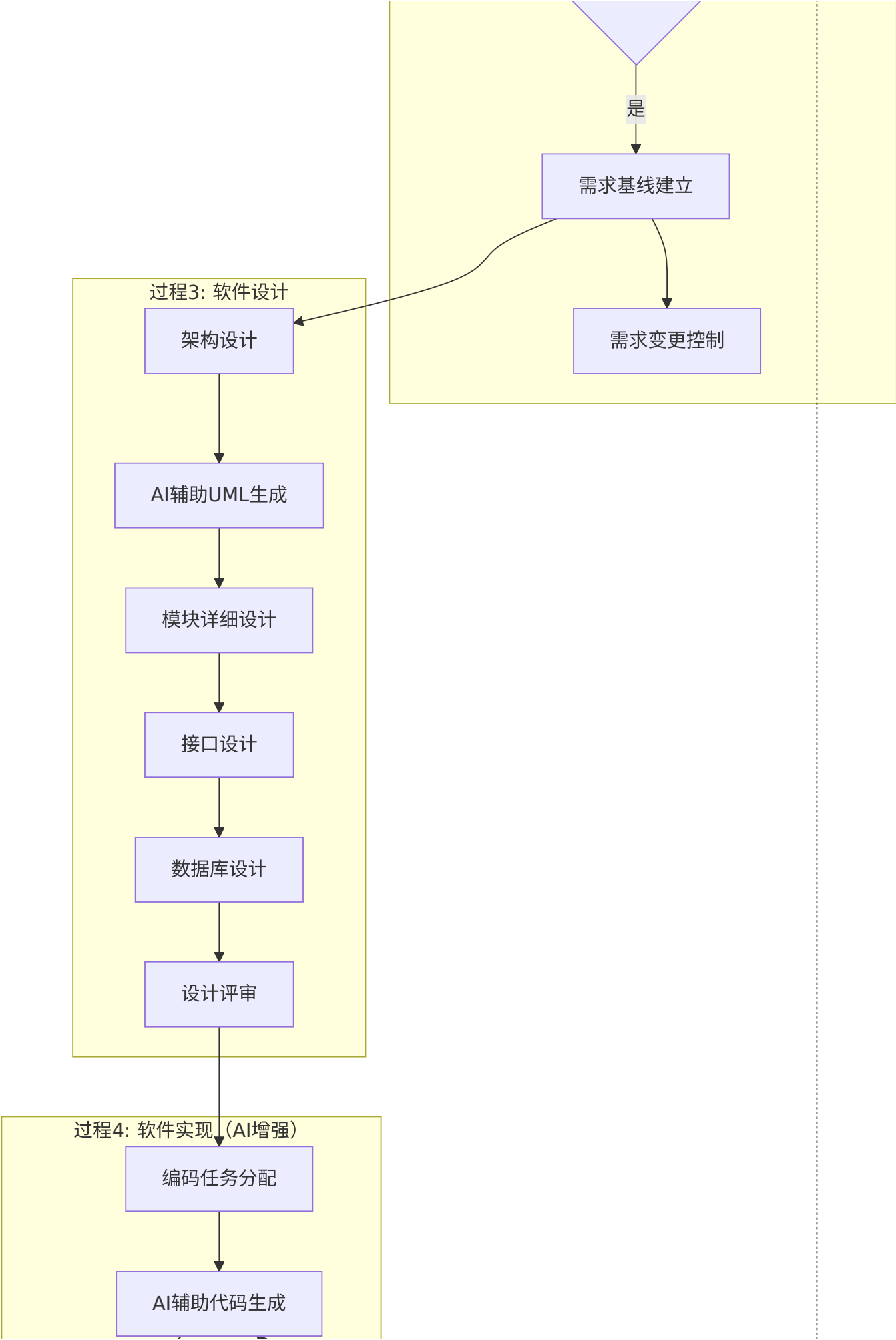
3.1 全景流程图

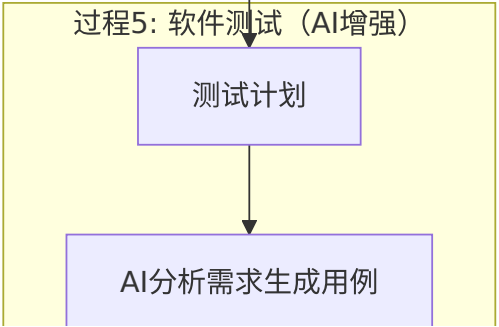
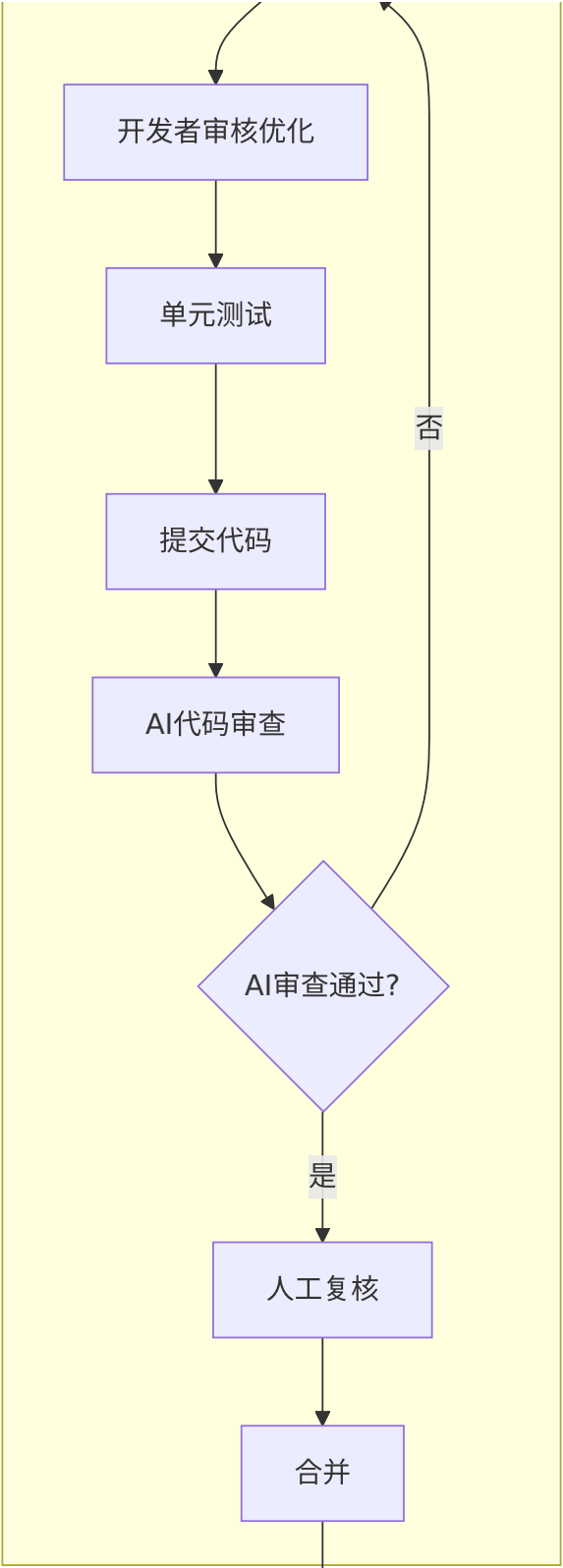
过程1: 项目启动与规划



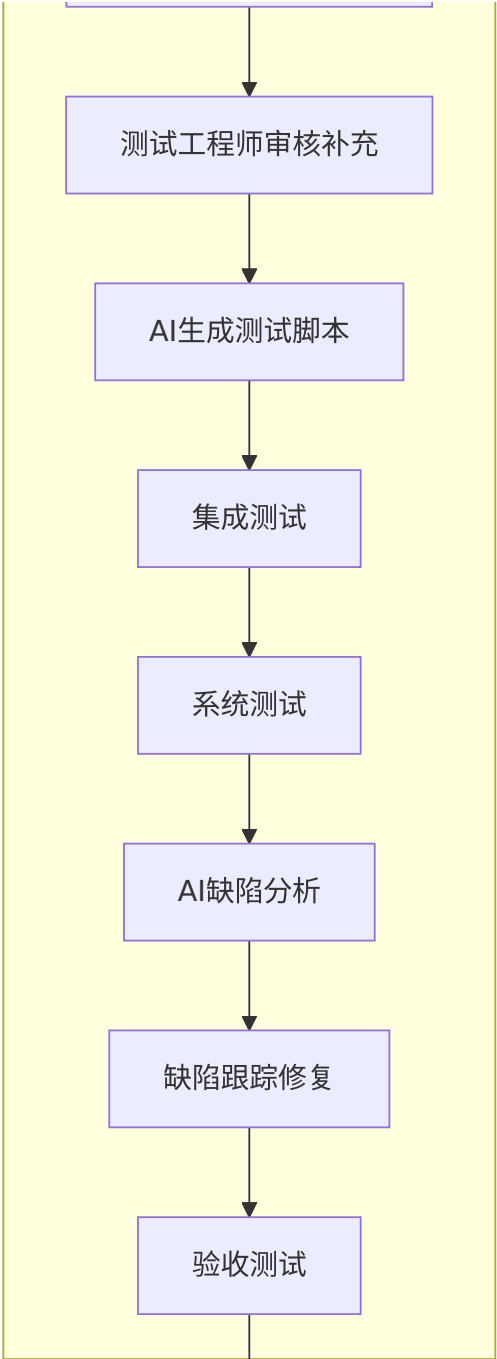
过程2: 需求工程 (AI增强)



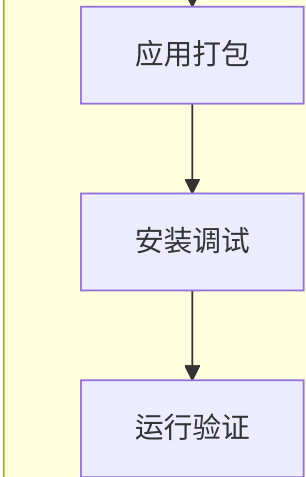


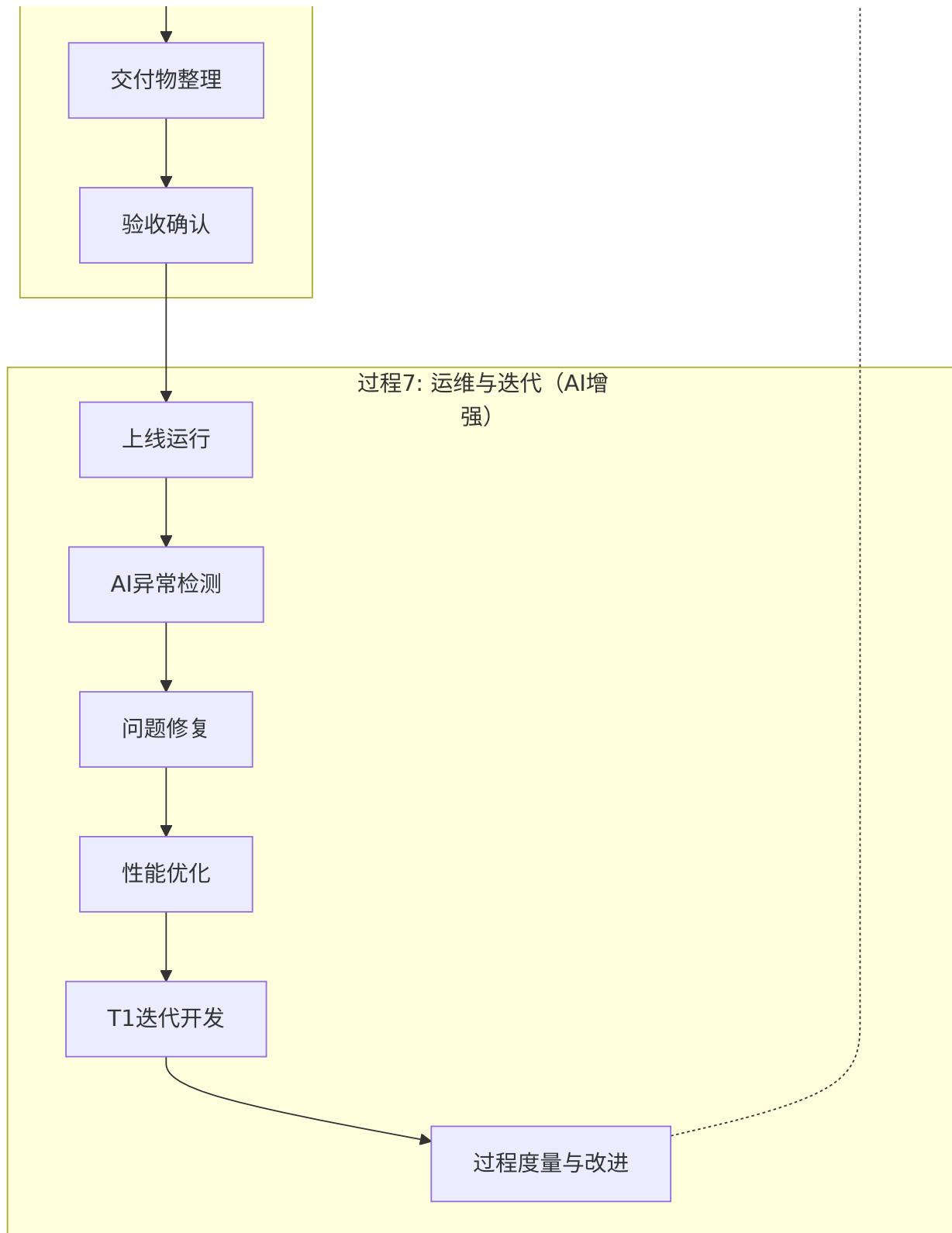


反馈



过程6: 部署与交付





3.2 AI节点详细说明

在需求工程过程中设有两个AI节点。节点1为AI辅助需求文档生成，其输入包括用户访谈记录、用户故事和会议纪要，使用的AI工具为ChatGPT或Claude。该节点的活动描述为：将非结构化需求输入提交至大语言模型，AI自动生成符合ISO 29148标准的结构化需求文档，包含功能需求、非功能需求和约束条件，输出为需求文档初稿（Markdown格式）。质量标准要求覆盖率不低于95%，每千字错误不超过3处。节点2为AI一致性校验，其输入为需求文档初稿，使用的AI工具为大语言模

型配合Prompt模板。该节点自动检测需求的冲突（矛盾需求）、冗余（重复需求）、遗漏（缺失场景）和可测试性问题，输出为一致性校验报告，冲突检测率要求不低于90%。

在软件设计过程中设有一个AI节点。节点3为AI辅助UML生成，其输入包括需求规格说明书和架构决策记录，使用的AI工具为ChatGPT或Mermaid AI。该节点根据需求描述自动生成UML类图、时序图和架构图的Mermaid代码，经人工调整后纳入设计文档，输出为UML设计图（Mermaid格式）。

在软件实现过程中设有两个AI节点。节点4为AI辅助代码生成，其输入包括详细设计文档和接口定义，使用的AI工具为Cursor或GitHub Copilot。开发者在IDE中使用AI Agent或代码补全功能生成代码骨架和核心业务逻辑，AI根据上下文理解项目结构并生成符合项目风格的一致代码，输出为源代码（含AI生成部分标记）。质量标准要求零高危静态分析问题且代码风格100%一致。节点5为AI代码审查，其输入为提交的代码变更（Pull Request），使用的AI工具为CodeRabbit。在CI流水线中自动触发AI代码审查，提供逐行反馈，运行40余种linter和安全扫描器，生成审查摘要，输出为AI代码审查报告，审查覆盖率要求不低于90%的修改文件。

在软件测试过程中设有三个AI节点。节点6为AI分析需求生成用例，其输入为需求基线文档，使用的AI工具为GPT-4o或Diffblue。AI自动解析需求文档，按等价类划分和边界值分析法生成功能测试用例，覆盖正常、异常和边界场景，输出为测试用例集（Markdown格式）。质量标准要求需求覆盖率不低于95%，异常场景占比不低于30%。节点7为AI生成测试脚本，其输入为测试用例集，使用的AI工具为Diffblue或通义灵码。AI将自然语言描述的测试用例自动转化为可执行的测试脚本（ArkTS或Python），集成到测试框架中，首次执行成功率要求不低于80%。节点8为AI缺陷分析，其输入包括测试执行日志和失败用例，使用的AI工具为SonarQube AI分析或ChatGPT。AI自动分析测试失败原因，定位缺陷根因，生成结构化缺陷报告，包含复现步骤、影响范围和建议修复方案，缺陷分析准确率要求不低于85%。

在运维与迭代过程中设有一个AI节点。节点9为AI异常检测，其输入包括运行监控日志和性能指标，使用的AI工具为自研AI异常检测模型或开源方案。AI实时分析应用运行数据，识别异常模式如CPU突增、响应延迟和错误率上升，触发告警并给出初步诊断，输出为异常告警通知和诊断信息。

第4章 交付物清单与质量标准

4.1 交付物总清单

过程	交付物	形式	AI参与	负责人
P1	项目计划书	Markdown	否	项目经理
P1	风险管理表	Markdown	否	项目经理
P2	需求规格说明书	Markdown	AI生成+人工优化	需求分析师
P2	需求一致性校验报告	Markdown	AI生成	AI训练师
P2	用户故事地图	Mermaid	否	需求分析师
P3	系统架构设计文档	Markdown	否	架构师
P3	UML设计图	Mermaid	AI辅助生成	架构师
P3	接口定义文档	Markdown	否	开发工程师
P3	数据库设计文档	Markdown	否	开发工程师
P4	源代码（含AI生成标记）	ArkTS/TS	AI辅助生成	开发工程师
P4	AI代码审查报告	Markdown	AI生成	AI输出审核员

P4	单元测试报告	Markdown	否	开发工程师
P5	测试计划	Markdown	否	测试工程师
P5	测试用例集	Markdown	AI生成+人工补充	AI测试分析师
P5	测试脚本	ArkTS/Python	AI生成+人工审核	测试工程师
P5	缺陷报告	Markdown	AI辅助分析	测试工程师
P5	测试总结报告	Markdown	否	测试工程师
P6	应用安装包	.hap	否	开发工程师
P6	用户使用手册	Markdown	否	项目经理
P6	部署说明文档	Markdown	否	运维工程师
P7	运维日志分析报告	Markdown	AI辅助	运维工程师
P7	迭代计划	Markdown	否	项目经理

4.2 AI相关交付物质量标准

在AI生成的代码方面，核心质量指标包括三项。静态分析通过率要求达到100%，即零Critical和Severe问题，通过CI自动扫描验证。风格一致性要求100%匹配eslint配置，通过CI自动检查验证。人工审核通过率要求不低于95%，通过人工Review验证。

在AI生成的文档方面，事实准确性错误率要求不超过每千字3处，通过人工抽查验证。

在AI生成的测试用例方面，需求覆盖率要求不低于95%，通过追溯矩阵验证。异常场景占比要求不低于30%，通过分类统计验证。

在AI生成的测试脚本方面，首次执行成功率要求不低于80%，通过自动执行验证。

在AI缺陷分析方面，缺陷根因定位准确率要求不低于85%，通过人工确认验证。

第5章 过程度量指标（含AI贡献度指标）

5.1 传统过程度量指标

在进度指标方面，需求评审周期指从需求提交到基线建立的天数，目标值为不超过5天。编码周期指从编码开始到提测的天数，目标值为不超过15天。测试周期指从提测到验收通过的天数，目标

值为不超过10天。

在质量指标方面，缺陷密度指每千行代码的缺陷数，目标值为不超过5个。需求覆盖率指测试用例对需求条目的覆盖率，目标值不低于95%。发布前缺陷检出率指发布前发现缺陷占总缺陷的比例，目标值不低于90%。

在效率指标方面，代码评审效率指每百行代码的评审耗时，目标值为不超过30分钟。

5.2 AI贡献度指标（新增）

指标名称	定义	计算方法	目标值
AI代码生成率	衡量AI对编码效率的贡献	$AI代码行数 / 总代码行数 \times 100\%$	30%至50%
AI审查覆盖率	衡量AI代码审查的普及度	$AI审查PR数 / 总PR数 \times 100\%$	100%
AI审查采纳率	衡量AI审查建议的质量	$采纳建议数 / 总建议数 \times 100\%$	不低于70%
AI测试用例采纳率	衡量AI测试用例质量	$通过审核用例数 / AI总用例数 \times 100\%$	不低于85%
AI缺陷预测命中率	衡量AI缺陷预测准确度	$命中数 / 预测数 \times 100\%$	不低于60%
AI工具可用率	衡量AI工具基础设施稳定性	$可用时间 / 总时间 \times 100\%$	不低于99%
人机协同效率比	衡量AI引入的综合效率提升	$AI辅助耗时 / 纯人工耗时 \times 100\%$	不超过50%
AI输出错误率	衡量AI输出质量	$需人工修正项数 / 总输出项数 \times 100\%$	不超过15%

5.3 度量数据采集方式

度量数据的采集分为自动采集和人工录入两类。自动采集的数据来源包括Git版本控制系统（提供代码行数和提交频率）、CI/CD流水线（提供审查覆盖率和采纳率）、测试管理系统（提供用例数和覆盖率）以及AI工具日志（提供生成量和可用率）。人工录入的数据包括项目周报中的手工填写数据和里程碑评审中的阶段数据。所有数据汇集到度量数据库后，经过数据分析生成度量仪表盘，最终为项目管理提供改进决策支持。

5.4 持续改进机制

本过程采用PDCA循环进行持续改进。在计划阶段，每两周基于度量数据识别过程中存在的瓶颈和改进点。在执行阶段，制定改进措施并执行，例如优化Prompt模板或调整审核策略。在检查阶段，在下一度量周期验证改进措施的有效性。在处置阶段，将验证有效的改进措施固化为标准流程，无效措施则重新分析原因。此外，每月进行AI专项改进评估，评估AI工具的使用效果并更新Prompt模板库和最佳实践文档。

第6章 AI工具集成指南

6.1 工具部署架构

本地开发环境的部署方案为：开发者在Cursor IDE中进行编码，IDE集成了GitHub Copilot用于AI代码补全，同时通过Ollama运行本地CodeLLaMA模型用于处理敏感代码。云端服务层通过LLM API Gateway统一管理GPT-4o和Claude等大语言模型的调用。CodeRabbit Cloud通过Webhook与GitHub仓库集成，在Pull Request提交时自动触发代码审查。CI/CD流水线包括Jenkins进行持续集成构建、SonarQube加AI插件进行质量门禁检查，以及测试执行器运行AI生成的测试脚本。

6.2 工具配置清单

工具	版本/计划	配置要点	成本类型
Cursor IDE	最新版	启用Agent模式，配置项目上下文	免费版或Pro版
GitHub Copilot	Pro版	为团队成员开通，配置代码排除规则	付费
CodeRabbit	Free或Team版	连接到GitHub仓库，配置审查规则	免费或付费
ChatGPT/Claude	Web端	准备结构化Prompt模板库	免费或付费
Diffblue Cover	Community版	配置Maven或Gradle集成	社区版免费
SonarQube	Community版+AI插件	配置质量门禁规则	免费
Ollama	最新版	本地部署，用于敏感代码处理	免费开源

第7章 改进总结

本次过程改进以HarmonyWardrobe项目为载体，系统性地将AI技术融入软件全生命周期。在过程层面，在3个核心过程中新增9个AI参与节点，覆盖从需求分析到代码生成、代码审查、测试生成、缺陷分析再到运维监控的全链路。在角色层面，新增3个AI相关角色，为人机协同提供了组织保障。在质量层面，建立了AI输出物的双向质量标准，同时设定了AI生成的质量要求和人工审核的验证机制。在度量层面，新增8项AI贡献度指标，使AI改进效果可量化、可追踪。在工具层面，完成了从本地IDE到云端CI/CD的完整AI工具链部署方案设计。

通过以上改进，预期可达成需求文档编写效率提升50%、编码效率提升30%至40%、测试用例设计效率提升60%至80%的综合改进目标。

参考文献

- [1] ISO/IEC 12207:2017, Systems and software engineering — Software life cycle processes.
- [2] ISO/IEC 15504 (SPICE), Information technology — Process assessment.
- [3] ISO/IEC 33001:2015, Information technology — Process assessment — Concepts and terminology.
- [4] Deloitte, "The Inflection Point: Agentic AI in Software Engineering — From SDLC to AO-DLC," May 2026.
- [5] H. Cheng et al., "Generative AI for Requirements Engineering: A Systematic Literature Review," *Software: Practice and Experience*, Nov. 2025.
- [6] 中国信通院, 《智能化软件工程技术和应用要求 第3部分：智能测试能力》, 2024.
- [7] AI4SE 行业现状调查报告, 2024.
- [8] A. Seelamneni, "AI in QA: Transforming Test Automation and Software Quality Through Intelligent Solutions," *WJAETS*, 2025.
- [9] J. Schiesel, "Beyond Code Generation: How AI Is Expanding Across the Entire SDLC," Metacto, Apr. 2026.
- [10] K. Alge, "The Great Decoupling: AI in Software Development," LOOM White Paper, Feb. 2026.